Entrega C — Nginx comentado e fluxo de uma requisição (Hora 4)

Arquivo: `nginx/nginx.conf` — cada diretiva está comentada no próprio arquivo. Base: Apêndice A.2. Ajustes nossos marcados `[AJUSTE]` no arquivo.

1. Diretivas e por que cada valor

Diretiva	Valor	Por quê
`log_format sentinel`	`request_id`, `upstream`, `node`, `upstream_status`, `req_time`, `upstream_time`, `msec`	Correlacionar com o Rust e ver retry/tempos por tentativa. `msec` dá milissegundos
`upstream rust_pool` (sem algoritmo)	round-robin	Revezamento A, B, A, B — fácil de provar com 20 requisições
`max_fails=3 fail_timeout=10s`	3 falhas em 10 s → fora por 10 s	Não tira o nó por uma falha isolada; volta a tentar depois de 10 s. Verificação **passiva**
`zone` + `resolve` + `resolver 127.0.0.11 valid=10s`	[AJUSTE]	Re-resolve os nomes a cada 10 s: se o contêiner voltar com outro IP, o Nginx o acha sem reload. **Efeito colateral medido**: o tráfego leva até ~10 s para voltar após o `start` (≤ 4,6 s, 9,4 s, 10,5 s; achado 4)
`proxy_set_header X-Request-Id $request_id`		Propaga o id ao nó
`proxy_connect_timeout 1s`		Nó parado é detectado em 1 s. Na rede local, conectar leva < 1 ms: 1 s é folga de 1000×
`proxy_read_timeout 3s` / `proxy_send_timeout 3s`		Um nó lento de 2,5 s ainda responde (E2); acima de 3 s vira falha e retry (E2b)
`proxy_next_upstream error timeout http_502 http_503`		Refaz **só** falhas que indicam "este nó não pode agora". **Não** refaz 500 (bug da aplicação: repetir só dobra a carga)
`proxy_next_upstream_tries 2`		Original + 1 no outro nó. Com 2 nós, mais que isso só voltaria ao mesmo nó
`proxy_next_upstream_timeout 4s`		Teto de tempo gasto em retries
`proxy_hide_header X-Request-Id` + `add_header X-Request-Id/X-Upstream`	[AJUSTE]	Cliente vê o id e o nó sem abrir logs; evita o cabeçalho duplicado
`location = /nginx/live`	[AJUSTE]	Liveness do **próprio** Nginx, sem depender dos nós (achado 6)
`location /admin/` e `/metrics` → 404		Nada administrativo pela porta pública
`server_tokens off`		Não expõe a versão

2. Fluxo de uma requisição (base do item C)

Diagrama (fonte Mermaid — renderizado no GitHub/VS Code):

```
sequenceDiagram
  autonumber
  participant C as Cliente (curl)
  participant N as Nginx :80
  participant A as rust-a:8080
  participant B as rust-b:8080
  C->>N: GET / (host:8080)
  Note over N: gera request_id; escolhe nó por round-robin
  N->>A: GET / + X-Request-Id
  alt A responde em < 3 s
    A-->>N: 200 "atendido por rust-a" (loga request_id)
  else A não conecta em 1 s / 502 / 503
    N->>B: retry (tentativa 2 de 2), mesmo request_id
    B-->>N: 200 "atendido por rust-b"
    Note over N: conta 1 falha para A (3 em 10 s ⇒ A fora por 10 s)
  end
  N-->>C: resposta + X-Request-Id + X-Upstream
  Note over N: loga status, upstream(s), upstream_status, tempos
```

1. O cliente envia `GET /` para `localhost:8080` (porta publicada → `nginx:80`).
2. O Nginx gera um `request_id` (32 hex), escolhe o próximo nó do `rust_pool` em revezamento e repassa com `X-Request-Id`.
3. O nó Rust atende, registra o `request_id` no log e responde (com `X-Node`).
4. O Nginx devolve a resposta e registra o nó usado, os status e os tempos.
5. Se o nó não conecta em 1 s, demora mais de 3 s ou responde 502/503, o Nginx tenta **uma vez** o outro nó. 500 não é refeito.

3. Evidências da Hora 4

Passo do enunciado	Arquivo	Resultado
20 requisições, contar por nó	`evidencias/preliminar/05_no_parado_e_recuperacao.txt` (bloco final) e `05b`	A 10 / B 10 (em `04` deu A 0 / B 20: A ainda estava fora do pool, ver achado 8)
Parar um nó e repetir	`05_no_parado_e_recuperacao.txt`, `05c`	A 0 / B 20, todas 200; em `05c`, 3 req a ~1,09 s (as que caíram em A antes da retirada); em `05`, nenhuma lenta (A já estava fora do pool, achado 8)
Quebrar a config e ler o erro	`06_config_quebrada.txt`	ver seção 4
`request_id`, nó e tempos no log	`04_…` (fim) e `05_…`	mesmo id no Nginx e no Rust

Contagem (Hora 4, quadro do enunciado):

Rust A	Rust B	Com A parado: A	B
10	10	0	20

4. Configuração quebrada de propósito

Testamos em **cópias** com `nginx -t` (não tocamos no Nginx em produção):

Quebra	**Resultado do `nginx -t`**	**Leitura**
Falta `;` em `proxy_connect_timeout 1s`	erro de sintaxe apontando arquivo e linha	o Nginx recusa iniciar/recarregar → o serviço antigo continua de pé se usar `nginx -s reload`
`server rust-c:8080` (host inexistente, sem `resolve`)	`host not found in upstream "rust-c:8080"`	sem `resolve` o nome tem de existir **na inicialização** — um nó ausente impediria o Nginx inteiro de subir
`proxy_pass http://pool_errado`	`host not found in upstream "pool_errado"`	upstream inexistente é tratado como nome de host

(Saídas literais em `06_config_quebrada.txt`.) Lição: **sempre `nginx -t` antes de `reload`**; e `resolve` também protege a inicialização quando um nó está fora.